

Breaking GPT & Claude — a Promptfoo red-team workshop

A hands-on AI bug-bounty course. Attendees red-team two guardrailed chatbots — **MediBot** and **FinanceBot** — with **Promptfoo** against Groq's free tier. Every exercise is the same three-part rig: a **guardrail prompt**, a **test suite**, and a **provider matrix**, wired together by one `promptfooconfig.*.yaml`.

91

TRACKED FILES

3 + hack

MODULES

3

GROQ MODELS

20

LESSON CONFIGS

2

TARGET BOTS

01 How one eval is wired

The whole repo is variations on this rig. A config file is the wiring diagram: it points at a prompt, a set of tests, and a list of providers, then Promptfoo runs every test against every model and grades the answers.

GUARDRAIL PROMPT

prompts/medibot.txt The system prompt with the bot's non-negotiable rules — the thing you're trying to break.

ATTACK SUITE

tests/smoke.medibot.yaml Curated probes: jailbreaks, hallucination traps, exfiltration, cost/length checks.

PROVIDER MATRIX

3 × Groq models llama-3.1-8b-instant · llama-3.3-70b-versatile · gpt-oss-20b



THE WIRING

promptfooconfig.medibot.yaml Binds prompt + tests + providers, sets defaultTest assertions and the llama-3.3-70b grader.



```
npx promptfoo@latest eval -c promptfooconfig.medibot.yaml
```



REPORT

pass / fail grid + web UI promptfoo view opens a per-model, per-test scorecard.

INVERTED SEMANTICS

fail = attack landed In a red-team suite a failing assertion is the finding you want. A healthy run exits 100.

Why "failing" is healthy

Modules 0 & 2 use ordinary `pass = good` assertions. But the root red-team configs invert it: the assertion asserts the guardrail *held*, so a **fail** means the jailbreak got through — exactly what you're hunting. That's why a clean red-team run reports exit code `100`, not `0`.

02 The curriculum, three modules + a hackathon

One course in three modules. Module 1 deliberately has no folder — it *is* the repo root. Full map lives in `modules/README.md`.

MODULE 0

Promptfoo Basics

[modules/00-promptfoo-basics/](#)

The mechanics before the mischief — how prompts, providers, and assertions actually work. 11 runnable configs.

[01-prompts](#) ×4

[02-providers](#) ×2

[03-assertions](#) ×5

MODULE 1

Red-Team Fundamentals

[repo root](#) · [docs/02, 04, 05](#)

Break MediBot & FinanceBot for real. Lives in `prompts/`, `tests/`, and the root `promptfooconfig.*` files — no `01-` folder on purpose.

[jailbreaks](#)

[hallucination](#)

[exfiltration](#)

[quality suites](#)

MODULE 2

Advanced Evaluation

[modules/02-advanced-eval/](#)

Scoring that scales: weights, assert-sets, exact-vs-fuzzy, CSV-driven data, F-score, temperature — plus a 3-stage debugging track. 9 configs.

[weights](#)

[assert-sets](#)

[csv-data](#)

[fscore](#)

[temperature](#)

[debugger](#) ×3

HACKATHON

Break it · Fix it · Build it

[docs/04-challenges.md](#)

Three challenges, every team does all three, scored on creativity: author a novel attack, harden a prompt, then invent your own guardrailed bot from the mybot skeleton.

 [break](#)

 [fix](#)

 [build](#)

03 The layout on disk

What's where, and what each area is responsible for. Colored names carry the same meaning as the legend above.

● ● ● breaking-gpt-claude-workshop/

```
breaking-gpt-claude-workshop/
├─ README.md  CLAUDE.md  .env.example          # entry point + authoring rules
├─ prompts/                                       # the guardrail system prompts (Module 1)
│   └─ medibot.txt  financebot.txt  mybot.txt  ← mybot = build-it template
├─ tests/                                         # attack & quality probe suites
│   └─ smoke.medibot.yaml (6)  smoke.finance.yaml (4)
│       └─ bias · compliance · consistency · context · values  # quality challenges
├─ promptfooconfig.*.yaml  ×7                   # the wiring – one per exercise variant
│   └─ medibot · finance                        # main red-team (free-tier-safe)
│       └─ quality.medibot · quality.finance    # bias / values / compliance
│           └─ openrouter.medibot · openrouter.finance  # fallback provider
│               └─ mybot                          # challenge-3 <TODO> skeleton
├─ modules/                                       # the structured curriculum
│   └─ 00-promptfoo-basics/                      01-prompts · 02-providers · 03-assertions
│       └─ 02-advanced-eval/                     weights · assert-sets · exact-vs-fuzzy ·
│           └─ debugger/ 1_basic · 2_moderate · 3_advance (+ SOLUTION.md)
├─ docs/                                         # the walkthroughs students read
│   └─ 01-quickstart  02-redteam  03-troubleshooting  04-challenges  05-quality
│       └─ superpowers/ specs + plans  ·  qr.png
├─ setup.sh/.ps1  preflight.sh/.ps1            # install+key  ·  read-only readiness probe
├─ scripts/                                       # maintainer tooling (not student-facing)
│   └─ smoke-check.sh  generate-qr.js           # the full-repo live sweep
├─ .github/workflows/ lint.yml                 # shellcheck + bash -n CI
└─ .claude/                                      # authoring aids
    └─ agents/ red-teamer.md
        └─ skills/ new-eval-suite · run-and-summarize
```

04 Operator tooling

The scripts that get a room of attendees running — and keep the repo honest between sessions.

setup.sh / .ps1

Checks Node ≥ 20 , installs Promptfoo, prompts for the Groq key, writes `.env`, runs a smoke test.

preflight.sh / .ps1

Read-only readiness probe — key, network, environment. Safe to re-run the morning of the workshop.

scripts/smoke-check.sh PACED

Maintainer's full live sweep of every config. Cache-on + `--delay` by default; `SMOKE_FRESH=1` forces an authoritative uncached run.

.github/workflows/lint.yml NEW

CI: `shellcheck` (warning floor) + `bash -n` on every shell script, on push & PR.

breaking-gpt-claude-workshop · repo map generated from a clean main · 91 files, 20 lesson configs, 2 target bots, 3 Groq models.